JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DecimalFormatSymbols

java.lang.Object
    java.text.DecimalFormatSymbols

All Implemented Interfaces:

Serializable, Cloneable

```
public class DecimalFormatSymbols
extends Object
implements Cloneable, Serializable
```

This class represents the set of symbols (such as the decimal separator, the grouping separator, and so on) needed by `DecimalFormat` to format numbers. `DecimalFormat` creates for itself an instance of `DecimalFormatSymbols` from its locale data. If you need to change any of these symbols, you can get the `DecimalFormatSymbols` object from your `DecimalFormat` and modify it.

The "rg" (region override), "nu" (numbering system), and "cu" (currency) `Locale` Unicode extensions are supported which may override values within the symbols. For both "nu" and "cu", if they are specified in addition to "rg" by the backing `Locale`, the respective values from the "nu" and "cu" extension supersede the implicit ones from the "rg" extension.

Since:

1.1

See Also:

Locale, DecimalFormat, Serialized Form

Constructor Summary

Constructors

Constructor	Description
DecimalFormatSymbols()	Create a DecimalFormatSymbols object for the default FORMAT locale.
DecimalFormatSymbols(Locale locale)	Create a DecimalFormatSymbols object for the given locale.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description

Object	**clone**()	Standard override.
boolean	**equals**(Object obj)	Compares the specified object with this DecimalFormatSymbols for equality.
static Locale[]	**getAvailableLocales**()	Returns an array of all locales for which the getInstance methods of this class can return localized instances.
Currency	**getCurrency**()	Returns the Currency of this DecimalFormatSymbols.
String	**getCurrencySymbol**()	Returns the currency symbol for the currency of these DecimalFormatSymbols in their locale.
char	**getDecimalSeparator**()	Gets the character used for decimal sign.
char	**getDigit**()	Gets the character used for a digit in a pattern.
String	**getExponentSeparator**()	Returns the string used to separate the mantissa from the exponent.
char	**getGroupingSeparator**()	Gets the character used for grouping separator.
String	**getInfinity**()	Gets the string used to represent infinity.
static final DecimalFormatSymbols	**getInstance**()	Gets the DecimalFormatSymbols instance for the default locale.
static final DecimalFormatSymbols	**getInstance** (Locale locale)	Gets the DecimalFormatSymbols

		instance for the specified locale.
String	**getInternationalCurrency**	Returns the ISO 4217 currency code of the currency of these DecimalFormatSymbols.
Locale	**getLocale**()	Returns locale used to create this instance.
char	**getMinusSign**()	Gets the character used to represent minus sign.
char	**getMonetaryDecimalSepara**	Returns the monetary decimal separator.
char	**getMonetaryGroupingSepar**	Gets the character used for grouping separator for currencies.
String	**getNaN**()	Gets the string used to represent "not a number".
char	**getPatternSeparator**()	Gets the character used to separate positive and negative subpatterns in a pattern.
char	**getPercent**()	Gets the character used for percent sign.
char	**getPerMill**()	Gets the character used for per mille sign.
char	**getZeroDigit**()	Gets the character used for zero.
int	**hashCode**()	Returns the hash code for this DecimalFormatSymbols.
void	**setCurrency**(Currency currency)	Sets the currency of this DecimalFormatSymbols.

void	**setCurrencySymbol**(String currency)	Sets the currency symbol for the currency of this DecimalFormatSymbols in their locale.
void	**setDecimalSeparator**(char decimalSeparator)	Sets the character used for decimal sign.
void	**setDigit**(char digit)	Sets the character used for a digit in a pattern.
void	**setExponentSeparator**(String exp)	Sets the string used to separate the mantissa from the exponent.
void	**setGroupingSeparator**(char groupingSeparator)	Sets the character used for grouping separator.
void	**setInfinity**(String infinity)	Sets the string used to represent infinity.
void	**setInternationalCurrency**(String currencyCode)	Sets the ISO 4217 currency code of the currency of these DecimalFormatSymbols.
void	**setMinusSign**(char minusSign)	Sets the character used to represent minus sign.
void	**setMonetaryDecimalSepara**(char sep)	Sets the monetary decimal separator.
void	**setMonetaryGroupingSepar**(char monetaryGroupingSe	Sets the character used for grouping separator for currencies.
void	**setNaN**(String NaN)	Sets the string used to represent "not a number".
void	**setPatternSeparator**(char patternSeparator)	Sets the character used to separate positive and negative subpatterns in a pattern.
void		

	setPercent (char percent)	Sets the character used for percent sign.
void	**setPerMill** (char perMill)	Sets the character used for per mille sign.
void	**setZeroDigit** (char zeroDigit)	Sets the character used for zero.

Methods declared in class Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Constructor Details

DecimalFormatSymbols

```
public DecimalFormatSymbols()
```

Create a DecimalFormatSymbols object for the default FORMAT locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed DecimalFormatSymbolsProvider implementations. For full locale coverage, use the getInstance method.

This is equivalent to calling DecimalFormatSymbols(Locale.getDefault(Locale.Category.FORMAT)).

See Also:

Locale.getDefault(java.util.Locale.Category), Locale.Category.FORMAT

DecimalFormatSymbols

```
public DecimalFormatSymbols(Locale locale)
```

Create a DecimalFormatSymbols object for the given locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed DecimalFormatSymbolsProvider implementations. For full locale coverage, use the getInstance method. If the specified locale contains the Locale.UNICODE_LOCALE_EXTENSION for the numbering system, the instance is initialized with the specified numbering system if the JRE implementation supports it. For example,

```
NumberFormat.getNumberInstance(Locale.forLanguageTag("th-TH-u-nu-thai"))
```

This may return a `NumberFormat` instance with the Thai numbering system, instead of the Latin numbering system.

Parameters:

`locale` - the desired locale

Throws:

`NullPointerException` - if `locale` is null

Method Details

getAvailableLocales

`public static Locale[] getAvailableLocales()`

Returns an array of all locales for which the `getInstance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `DecimalFormatSymbolsProvider` implementations. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

Returns:

an array of locales for which localized `DecimalFormatSymbols` instances are available.

Since:

1.6

getInstance

`public static final DecimalFormatSymbols getInstance()`

Gets the `DecimalFormatSymbols` instance for the default locale. This method provides access to `DecimalFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by installed `DecimalFormatSymbolsProvider` implementations.

This is equivalent to calling `getInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a `DecimalFormatSymbols` instance.

Since:

1.6

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getInstance

```
public static final DecimalFormatSymbols getInstance(Locale locale)
```

Gets the `DecimalFormatSymbols` instance for the specified locale. This method provides access to `DecimalFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by installed `DecimalFormatSymbolsProvider` implementations. If the specified locale contains the `Locale.UNICODE_LOCALE_EXTENSION` for the numbering system, the instance is initialized with the specified numbering system if the JRE implementation supports it. For example,

```
NumberFormat.getNumberInstance(Locale.forLanguageTag("th-TH-u-nu-thai"))
```

This may return a `NumberFormat` instance with the Thai numbering system, instead of the Latin numbering system.

Parameters:

`locale` - the desired locale.

Returns:

a `DecimalFormatSymbols` instance.

Throws:

`NullPointerException` - if `locale` is null

Since:

1.6

getLocale

```
public Locale getLocale()
```

Returns locale used to create this instance.

Returns:

locale used to create this instance

Since:

19

getZeroDigit

```
public char getZeroDigit()
```

Gets the character used for zero. Different for Arabic, etc.

Returns:

the character used for zero

setZeroDigit

```
public void setZeroDigit(char zeroDigit)
```

Sets the character used for zero. Different for Arabic, etc.

Parameters:

`zeroDigit` - the character used for zero

getGroupingSeparator

```
public char getGroupingSeparator()
```

Gets the character used for grouping separator. Different for French, etc.

Returns:

the grouping separator

setGroupingSeparator

```
public void setGroupingSeparator(char groupingSeparator)
```

Sets the character used for grouping separator. Different for French, etc.

Parameters:

`groupingSeparator` - the grouping separator

getDecimalSeparator

```
public char getDecimalSeparator()
```

Gets the character used for decimal sign. Different for French, etc.

Returns:

the character used for decimal sign

setDecimalSeparator

```
public void setDecimalSeparator(char decimalSeparator)
```

Sets the character used for decimal sign. Different for French, etc.

Parameters:

`decimalSeparator` - the character used for decimal sign

getPerMill

```
public char getPerMill()
```

Gets the character used for per mille sign. Different for Arabic, etc.

Returns:

the character used for per mille sign

setPerMill

```
public void setPerMill(char perMill)
```

Sets the character used for per mille sign. Different for Arabic, etc.

Parameters:

`perMill` - the character used for per mille sign

getPercent

```
public char getPercent()
```

Gets the character used for percent sign. Different for Arabic, etc.

Returns:

the character used for percent sign

setPercent

```
public void setPercent(char percent)
```

Sets the character used for percent sign. Different for Arabic, etc.

Parameters:

`percent` - the character used for percent sign

getDigit

```
public char getDigit()
```

Gets the character used for a digit in a pattern.

Returns:

the character used for a digit in a pattern

setDigit

```
public void setDigit(char digit)
```

Sets the character used for a digit in a pattern.

Parameters:

`digit` - the character used for a digit in a pattern

getPatternSeparator

```
public char getPatternSeparator()
```

Gets the character used to separate positive and negative subpatterns in a pattern.

Returns:

the pattern separator

setPatternSeparator

```
public void setPatternSeparator(char patternSeparator)
```

Sets the character used to separate positive and negative subpatterns in a pattern.

Parameters:

`patternSeparator` - the pattern separator

getInfinity

```
public String getInfinity()
```

Gets the string used to represent infinity. Almost always left unchanged.

Returns:

the string representing infinity

setInfinity

```
public void setInfinity(String infinity)
```

Sets the string used to represent infinity. Almost always left unchanged.

Parameters:

`infinity` - the string representing infinity

Throws:

`NullPointerException` - if `infinity` is `null`

getNaN

```
public String getNaN()
```

Gets the string used to represent "not a number". Almost always left unchanged.

Returns:

the string representing "not a number"

setNaN

```
public void setNaN(String NaN)
```

Sets the string used to represent "not a number". Almost always left unchanged.

Parameters:

`NaN` - the string representing "not a number"

Throws:

`NullPointerException` - if `NaN` is `null`

getMinusSign

```
public char getMinusSign()
```

Gets the character used to represent minus sign. If no explicit negative format is specified, one is formed by prefixing minusSign to the positive format.

Returns:

the character representing minus sign

setMinusSign

```
public void setMinusSign(char minusSign)
```

Sets the character used to represent minus sign. If no explicit negative format is specified, one is formed by prefixing minusSign to the positive format.

Parameters:

`minusSign` - the character representing minus sign

getCurrencySymbol

```
public String getCurrencySymbol()
```

Returns the currency symbol for the currency of these DecimalFormatSymbols in their locale.

Returns:

the currency symbol

Since:

1.2

setCurrencySymbol

`public void setCurrencySymbol(String currency)`

Sets the currency symbol for the currency of this `DecimalFormatSymbols` in their locale. Unlike `setInternationalCurrencySymbol(String)`, this method does not update the currency attribute nor the international currency symbol attribute.

Parameters:

`currency` - the currency symbol

Throws:

`NullPointerException` - if `currency` is `null`

Since:

1.2

getInternationalCurrencySymbol

`public String getInternationalCurrencySymbol()`

Returns the ISO 4217 currency code of the currency of these DecimalFormatSymbols.

Returns:

the currency code

Since:

1.2

setInternationalCurrencySymbol

`public void setInternationalCurrencySymbol(String currencyCode)`

Sets the ISO 4217 currency code of the currency of these DecimalFormatSymbols. If the currency code is valid (as defined by `Currency.getInstance`), this also sets the currency attribute to the corresponding Currency instance and the currency symbol attribute to the currency's symbol in the DecimalFormatSymbols' locale. If the currency code is not valid, then the currency attribute and the currency symbol attribute are not modified.

Parameters:

`currencyCode` - the currency code

Throws:

`NullPointerException` - if `currencyCode` is `null`

Since:

1.2

See Also:

`setCurrency(Currency)`, `setCurrencySymbol(String)`

getCurrency

```
public Currency getCurrency()
```

Returns the `Currency` of this `DecimalFormatSymbols`.

Returns:

the `Currency` of this `DecimalFormatSymbols`

Since:

1.4

setCurrency

```
public void setCurrency(Currency currency)
```

Sets the currency of this `DecimalFormatSymbols`. This also sets the currency symbol attribute to the currency's symbol in the DecimalFormatSymbols' locale, and the international currency symbol attribute to the currency's ISO 4217 currency code.

Parameters:

`currency` - the new currency to be used

Throws:

`NullPointerException` - if `currency` is null

Since:

1.4

See Also:

`setCurrencySymbol(String)`,
`setInternationalCurrencySymbol(String)`

getMonetaryDecimalSeparator

```
public char getMonetaryDecimalSeparator()
```

Returns the monetary decimal separator.

Returns:

the monetary decimal separator

Since:

1.2

setMonetaryDecimalSeparator

```
public void setMonetaryDecimalSeparator(char sep)
```

Sets the monetary decimal separator.

Parameters:

`sep` - the monetary decimal separator

Since:

1.2

getExponentSeparator

```
public String getExponentSeparator()
```

Returns the string used to separate the mantissa from the exponent. Examples: "x10^" for 1.23x10^4, "E" for 1.23E4.

Returns:

the exponent separator string

Since:

1.6

See Also:

`setExponentSeparator(java.lang.String)`

setExponentSeparator

```
public void setExponentSeparator(String exp)
```

Sets the string used to separate the mantissa from the exponent. Examples: "x10^" for 1.23x10^4, "E" for 1.23E4.

Parameters:

`exp` - the exponent separator string

Throws:

`NullPointerException` - if `exp` is null

Since:

1.6

See Also:

`getExponentSeparator()`

getMonetaryGroupingSeparator

```
public char getMonetaryGroupingSeparator()
```

Gets the character used for grouping separator for currencies. May be different from `grouping separator` in some locales, e.g, German in Austria.

Returns:

the monetary grouping separator

Since:

15

setMonetaryGroupingSeparator

```
public void setMonetaryGroupingSeparator
(char monetaryGroupingSeparator)
```

Sets the character used for grouping separator for currencies. Invocation of this method will not affect the normal `grouping separator`.

Parameters:

`monetaryGroupingSeparator` - the monetary grouping separator

Since:

15

See Also:

`setGroupingSeparator(char)`

clone

```
public Object clone()
```

Standard override.

Overrides:

`clone` in class `Object`

Returns:

a clone of this instance.

See Also:

`Cloneable`

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this `DecimalFormatSymbols` for equality. Returns true if the object is also a `DecimalFormatSymbols` and the two `DecimalFormatSymbols` objects represent the same set of symbols.

Overrides:

`equals` in class `Object`

Implementation Requirements:

This method performs an equality check with a notion of class identity based on `getClass()`, rather than `instanceof`. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

`obj` - object to be compared for equality

Returns:

`true` if the specified object is equal to this `DecimalFormatSymbols`

See Also:

`Object.equals(Object)`

hashCode

```
public int hashCode()
```

Returns the hash code for this `DecimalFormatSymbols`.

Overrides:

`hashCode` in class `Object`

Implementation Requirements:

Non-transient instance fields of this class are used to calculate a hash code value which adheres to the contract defined in `Objects.hashCode(Object)`.

Returns:

the hash code for this `DecimalFormatSymbols`

See Also:

`Object.hashCode()`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.